HEALED: Hybrid Homomorphic Encryption for Analysis of Large-Scale Encrypted Data

Maria João Dias[a], Ivan Costa[(✉)b], Ivone Amorim[c], Eva Maia[d] and Isabel Praça[e]
GECAD, ISEP, Polytechnic of Porto, 4249-015 Porto, Portugal

Keywords: Hybrid Homomorphic Encryption, Transciphering, Privacy-Preserving Machine Learning, Healthcare, Medical Diagnosis, CKKS, BFV, Rubato.

Abstract: Integrating Machine Learning into healthcare raises critical concerns related to data privacy and, at the same time, processing large volumes of data is limited by regulations for sensitive information handling. Homomorphic Encryption (HE) overcomes such limitations by enabling computations on encrypted data but its heavy computational costs limit practical adoption. This paper explores Hybrid Homomorphic Encryption (HHE) as a practical solution for privacy in healthcare and proposes HEALED, a novel framework for medical diagnosis using the symmetric cipher Rubato, and the CKKS and BFV HE schemes. Experimental evaluation demonstrates that HEALED improves diagnostic accuracy compared to both the plaintext implementation and a CKKS baseline (plaintext: 95.43%, baseline: 94.21%, HEALED: 97.48 – 97.90%). Furthermore, it reduces encrypted data size by a factor of 4370× and reduces client-side computations, namely encryption time is reduced at least 62× and decryption time by 76×, when compared to the CKKS baseline. Security and performance trade-offs were also analyzed under varying HE parameters, showing that the system balances efficiency with strong confidentiality guarantees.

1 INTRODUCTION

The use of Artificial Intelligence (AI) and Machine Learning (ML) has increased in many areas, including sensitive domains such as healthcare, transforming medicine through applications in disease diagnosis, treatment planning, and monitoring (Bajwa et al., 2021). These typically outsource computations to cloud servers, where the environment may not be trusted by the data owners (Huang and Fang, 2024), introducing a constant need to guarantee data privacy. Data protection laws are essential to protect personal data, but can restrict access for AI applications and hinder their integration in critical fields (Saw and Ng, 2022). In this context, Privacy-Preserving Machine Learning (PPML) has emerged, proposing techniques that enable the use of sensitive data without compromising confidentiality. Several PPML techniques have been proposed, including Differential Privacy, Secure Multiparty Computation, and Trusted Execution Environments, each presenting practical limitations (Tanuwidjaja et al., 2020; Podschwadt et al., 2022). Another technique is Homomorphic Encryption (HE), a class of cryptographic schemes that performs computations directly over encrypted data, such that encrypted results correspond to equivalent operations on non-encrypted data (Acar et al., 2018). However, HE introduces significant computational overhead, limiting its large-scale adoption due to slow and heavy cryptographic operations (Gentry, 2009; Acar et al., 2018).

Hybrid Homomorphic Encryption (HHE) has been proposed as a compromise between security and efficiency, combining HE with more lightweight cryptographic methods, such as symmetric encryption (Cho et al., 2021; Dobraunig et al., 2023). HHE can be realized through *transciphering*, a process where sensitive data is encrypted with a symmetric scheme while the symmetric key is encrypted using HE and sent to an external computation party. This party homomorphically evaluates the symmetric decryption algorithm, transforming symmetric data into homomorphic, which can be privately processed (Abdinasibfar et al., 2025). Recent work

[a] https://orcid.org/0009-0006-5741-9103
[b] https://orcid.org/0009-0009-8480-9352
[c] https://orcid.org/0000-0001-6102-6165
[d] https://orcid.org/0000-0002-8075-531X
[e] https://orcid.org/0000-0002-2519-9859

Dias, M. J., Costa, I., Amorim, I., Maia, E. and Praça, I.
HEALED: Hybrid Homomorphic Encryption for Analysis of Large-Scale Encrypted Data.
DOI: 10.5220/0014357100004061
Paper published under CC license (CC BY-NC-ND 4.0)
In *Proceedings of the 12th International Conference on Information Systems Security and Privacy (ICISSP 2026) - Volume 2*, pages 252-259
ISBN: 978-989-758-800-6; ISSN: 2184-4356
Proceedings Copyright © 2026 by SCITEPRESS – Science and Technology Publications, Lda.

has explored transciphering-based PPML implementations in healthcare, including GuardML (Frimpong et al., 2024), SortingHat (Cong et al., 2022), and applications for Internet of Medical Things (IoMT) wearable devices (Huang and Duan, 2024). However, existing solutions do not provide end-to-end transciphering that simultaneously protects data, models, and results while reducing client-side overhead. In this context, this paper makes the following contributions:

1. A framework using Hybrid Homomorphic Encryption for Analysis of Large-scale Encrypted Data (HEALED) that builds upon the RtF-transciphering framework (Cho et al., 2021), leveraging the symmetric cipher Rubato (Ha et al., 2022) and the homomorphic schemes BFV (Fan and Vercauteren, 2012) and CKKS (Cheon et al., 2017)
2. An experimental healthcare PPML baseline using only HE which is based on Al Badawi et al.'s work (Al Badawi and Yusof, 2024).
3. An evaluation of the HEALED framework under varying HE parameters, with a comprehensive analysis of security and performance trade-offs.

2 BACKGROUND

This section introduces the essential concepts used in this work, including HE, HHE for efficient PPML, and the RtF-Transciphering framework that underpins our system.

2.1 Homomorphic Encryption

HE allows computations to be carried out over encrypted data (Acar et al., 2018), such that, after decryption, the results match the corresponding plaintext computations. HE uses $Eval()$, an evaluation algorithm specific to homomorphic schemes that applies operations on ciphertexts.

HE schemes differ in the supported operations and circuit depth. In this work, we focus on fully homomorphic schemes (FHE), which allow arbitrary computations (Acar et al., 2018). Common FHE schemes include BFV and BGV for exact arithmetic, TFHE for Boolean circuits, and CKKS for approximate arithmetic (Fan and Vercauteren, 2012; Brakerski et al., 2012; Chillotti et al., 2020; Cheon et al., 2017). These schemes incur significant overhead due to noise growth, bootstrapping, and ciphertext expansion, motivating hybrid approaches such as HHE.

2.2 Hybrid Homomorphic Encryption

HHE combines encrypted computation capabilities with more efficient cryptographic schemes, typically symmetric encryption, to reduce computational and communication overhead (Abdinasibfar et al., 2025). A common approach to HHE is *transciphering*, where data is encrypted symmetrically while the symmetric key is encrypted using HE, enabling encrypted computation on the data. Since transciphering requires homomorphic evaluation of the symmetric decryption algorithm, minimizing the multiplicative depth of these ciphers is essential to improve efficiency and often avoid bootstrapping (Abdinasibfar et al., 2025). These are usually HE-friendly ciphers, such as PASTA (Dobraunig et al., 2023), FiLIP (Méaux et al., 2019), HERA (Cho et al., 2021), and Rubato (Ha et al., 2022). To take advantage of its efficiency, transciphering is commonly applied in Client–Server applications, as illustrated in Figure 1. In these settings, the computationally heavy homomorphic operations are delegated to a server.

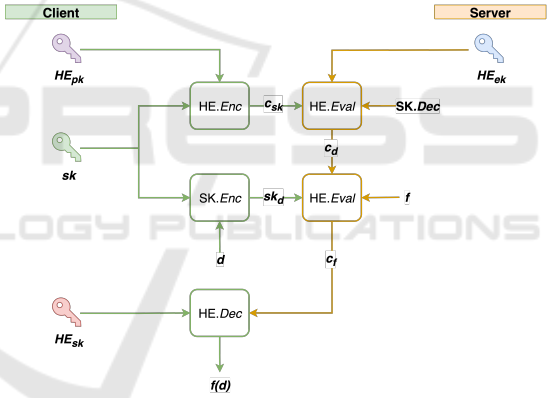


Figure 1: Simple client-server application with transciphering.

In transciphering, a client encrypts its data d using a symmetric key sk, generating sk_d, and encrypts the symmetric key using the homomorphic public key HE_{pk}. Both ciphertexts are sent to a server, which homomorphically evaluates the symmetric decryption algorithm to transform the symmetrically encrypted data into a homomorphic ciphertext. This ciphertext can then be evaluated with a function f, and the encrypted result is returned to the client for decryption with the homomorphic secret key.

2.3 RtF-Transciphering Framework

The *RtF-Transciphering framework* (Cho et al., 2021) is an instantiation of transciphering-based HHE. It

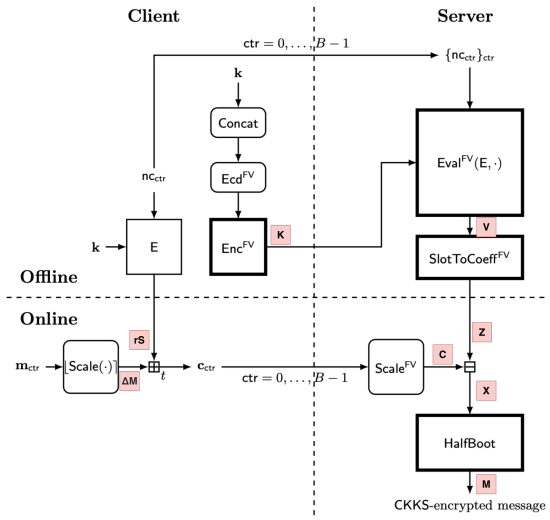


Figure 2: The RtF-Transciphering framework (from (Ha et al., 2022), red annotations added for clarity).

combines *BFV* (for exact arithmetic) and *CKKS* (for approximate arithmetic) with HE-friendly symmetric ciphers such as *HERA* and *Rubato*, enabling the efficient transformation of symmetrically encrypted data into a homomorphic domain. Figure 2 presents an overview of the framework, highlighting its *offline* and *online* phases.

In the offline phase, the client encrypts a symmetric key k under the BFV scheme and sends the resulting ciphertext to the server. The server homomorphically evaluates the symmetric cipher using this encrypted key, generating ciphertexts that encapsulate keystream values. These ciphertexts are then transformed to move the keystreams from slots to polynomial coefficients. During the online phase, the client first symmetrically encrypts its plaintext data and uploads the resulting ciphertext to the server. Using the precomputed encrypted keystreams, the server performs transciphering by removing the symmetric encryption homomorphically, obtaining a BFV ciphertext that encodes the scaled plaintext data. Finally, an adapted CKKS bootstrapping procedure, denoted as $HalfBoot$, is applied to convert the BFV ciphertext into a CKKS ciphertext, enabling approximate arithmetic for subsequent encrypted computations.

3 RELATED WORK

This section reviews the main healthcare PPML works that explicitly rely on transciphering. In GuardML (Frimpong et al., 2024), authors propose two systems that combine BFV with the HE-friendly cipher PASTA to reduce client-side costs in encrypted inference. In the basic configuration, the user symmetrically encrypts data and homomorphically encrypts the symmetric key; a cloud service provider performs transciphering and evaluates a neural network model. In a three-party configuration, an Analyst manages and encrypts model parameters, and may also decrypt results, reducing the user to symmetric encryption only. The reported accuracy is comparable to plaintext on the MIT-BIH dataset (Moody and Mark, 2001), but server-side computation and communication remain dominant bottlenecks.

SortingHat (Cong et al., 2022) explores transciphering with the FiLIP stream cipher alongside a private decision tree evaluation protocol optimized for homomorphic comparison. However, these components are not integrated into a single end-to-end pipeline, limiting applicability to practical healthcare PPML scenarios. Huang et al. (Huang and Duan, 2024) adapt SortingHat to multiple wearable IoMT devices by introducing a Trusted Third-Party (TTP) responsible for heavy setup and key sharing, while the computation server evaluates the decision tree. However, the transciphering integration remains unclear, model parameters are not protected, and results are decrypted outside the client side and communicated in plaintext.

Overall, existing transciphering-based healthcare PPML systems do not jointly provide (i) privacy for patient data, model parameters, and outputs, and (ii) low client-side burden through delegation of key generation and transciphering, while maintaining end-to-end confidentiality. Our paper addresses these gaps by proposing an HHE-based medical diagnosis workflow that keeps patient data, model parameters, and results encrypted throughout the pipeline.

4 HEALED

This section describes HEALED, outlining its system architecture and threat model..

4.1 System Model

HEALED follows a client–server architecture composed of two clients – the *Hospital* and the *Model Owner* – and two cloud-based servers – the *Trusted Third Party (TTP)* and the *Computing Server*, as shown in Figure 3. The Hospital holds sensitive patient data, while the Model Owner owns the trained ML model parameters. Both clients encrypt their data symmetrically, while the Computing Server performs transciphering and applies the ML model to the data

without access to plaintexts. The TTP is responsible for key generation and sharing, as well as encrypting the symmetric keys for transciphering.

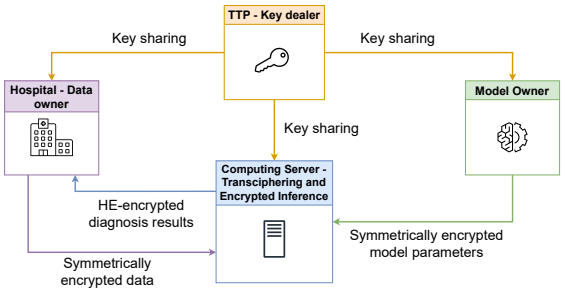


Figure 3: System overview for HEALED.

The protocol proceeds in four phases: *Initialization*, *Encryption*, *Evaluation*, and *Decryption*. During *Initialization*, the TTP generates the homomorphic public, secret, and evaluation keys $(HE_{pk}, HE_{sk}, HE_{ek})$ and two symmetric keys: sk_h and sk_m for the Hospital and Model Owner, respectively. The symmetric keys are encrypted under the homomorphic public key HE_{pk}, producing c_{sk_h} and c_{sk_m}. The Hospital receives the homomorphic secret key HE_{sk} and the symmetric key sk_h, the Model Owner receives its symmetric key sk_m, and the Computing Server receives the homomorphic evaluation key HE_{ek}, along with the encrypted symmetric keys. In the *Encryption* phase, the Hospital symmetrically encrypts patient data using sk_h, producing sk_{data}, while the Model Owner symmetrically encrypts model parameters using sk_m, producing sk_{param}. These ciphertexts are sent to the Computing Server. The *Evaluation* phase consists of transciphering followed by encrypted inference. First, the Computing Server homomorphically evaluates the symmetric decryption function using the encrypted keys, transforming symmetrically encrypted inputs into homomorphic ciphertexts:

$$c_{data} \leftarrow Eval(HE_{ek}, Dec_k, c_{sk_h}, sk_{data}), \quad (1)$$

$$c_{param} \leftarrow Eval(HE_{ek}, Dec_k, c_{sk_m}, sk_{param}). \quad (2)$$

These ciphertexts are the homomorphic ciphertexts of the patient data and model parameters. Next, encrypted diagnosis is performed by homomorphically evaluating the ML model function f over the resulting ciphertexts:

$$c_{res} \leftarrow Eval(HE_{ek}, f, c_{data}, c_{param}). \quad (3)$$

Finally, in the decryption phase, the Hospital decrypts c_{res} using the homomorphic secret key HE_{sk} to recover the plaintext diagnosis.

4.2 Threat Model

HEALED assumes an *honest-but-curious* adversarial model, where all entities correctly follow the prescribed protocol, but may attempt to infer sensitive information, such as patient data, model parameters, and diagnosis results. Malicious behavior from the entities, such as protocol deviation or collusion is out of scope. External adversaries are also considered; therefore, authenticated and encrypted communication channels are assumed for all plaintext communication, such as key and parameter distribution. These assumptions prevent eavesdropping and tampering during data transmission.

Under this model, no internal entity can access unauthorized plaintext information. The *Hospital* only accesses patient data and final decrypted results, but never the encrypted model parameters. The *Model Owner* only accesses its own model parameters and does not learn patient data or results. The *Computing Server* has access to all inputs required for evaluation, but exclusively in encrypted form, and does not possess decryption keys. The TTP is assumed to be fully trusted and does not behave maliciously. Its role is limited to cryptographic setup and key distribution, and it does not participate in data processing or inference.

5 IMPLEMENTATION

This section describes the prototype implementation of HEALED, developed in Go[1]. The system builds upon the RtF-Transciphering framework (Ha et al., 2022) and extends it to support encrypted inference of a linear SVM classifier, whose parameters are obtained from a model trained offline in Python.

5.1 The HEALED Prototype

HEALED adapts the RtF-Transciphering framework to a two-client, two-server architecture and extends it to support encrypted inference of a linear SVM classifier. Figure 4 illustrates the resulting framework and the distribution of the *Initialization*, *Encryption*, and *Evaluation* phases across entities.

During the *Initialization* phase, performed by the *TTP*, two symmetric keys and corresponding keystreams are generated: (key_h, rS_h) for the *Hospital* and (key_m, rS_m) for the *Model Owner*. Each symmetric key is encrypted under the BFV scheme, evaluated, and transformed using $SlotToCoeff^{FV}$, producing ciphertexts Z_h and Z_m, which are sent to the *Computing*

[1]https://go.dev.

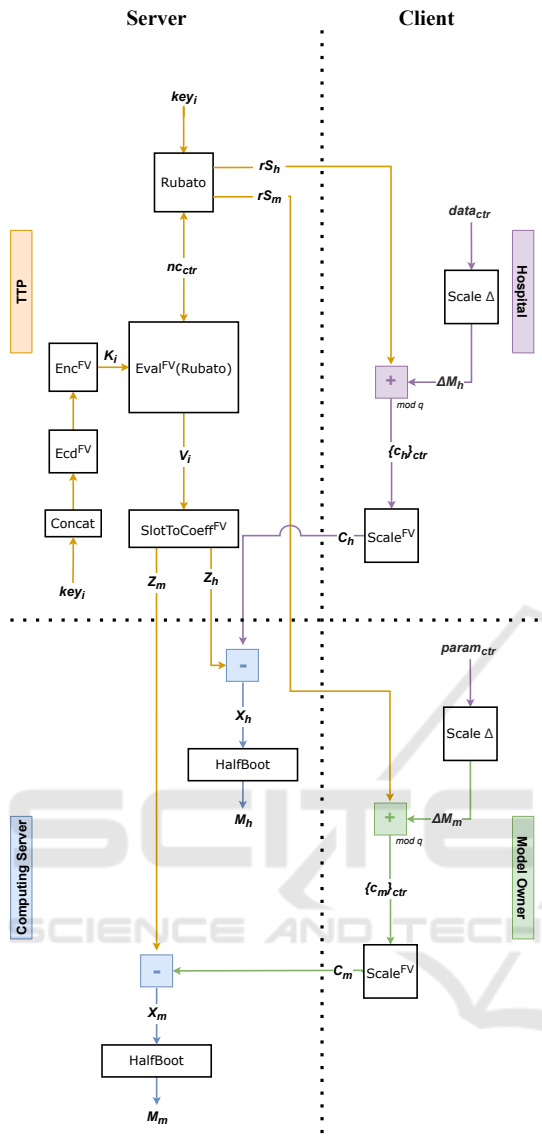


Figure 4: The HEALED transciphering framework after parameter setting and key generation/distribution. The figure uses $i \in \{h,m\}$ to simplify duplicated computations by the *TTP* for the *Hospital* and *Model Owner*.

Server. In the *Encryption* phase, both clients symmetrically encrypt their data using Rubato. The *Hospital* scales the patient data, adds the keystream rS_h, and maps the result to the BFV plaintext space. The *Model Owner* applies the same process to the model parameters using rS_m. The resulting ciphertexts C_h and C_m are sent to the *Computing Server*. The *Computing Server* performs transciphering by subtracting the encrypted keystreams from the received ciphertexts, yielding BFV ciphertexts of the scaled plaintexts. These ciphertexts are then processed by the $HalfBoot$ algorithm, producing CKKS ciphertexts $\mathcal{M}_h$ and $\mathcal{M}_m$ that encode patient data and model parameters, respectively.

Encrypted inference is illustrated in Figure 5, and is performed by evaluating the linear SVM score function $s(x) = \langle w,x \rangle + b$ over CKKS ciphertexts. The encrypted weight vector and bias are stored in $\mathcal{M}_m[0]$ and $\mathcal{M}_m[1]$. The *Computing Server* computes the encrypted inner product, followed by the addition of the bias, and returns the resulting ciphertext to the *Hospital*. Finally, the *Hospital* decrypts and decodes the result using the homomorphic secret key and determines the diagnosis based on the sign of the plaintext score.

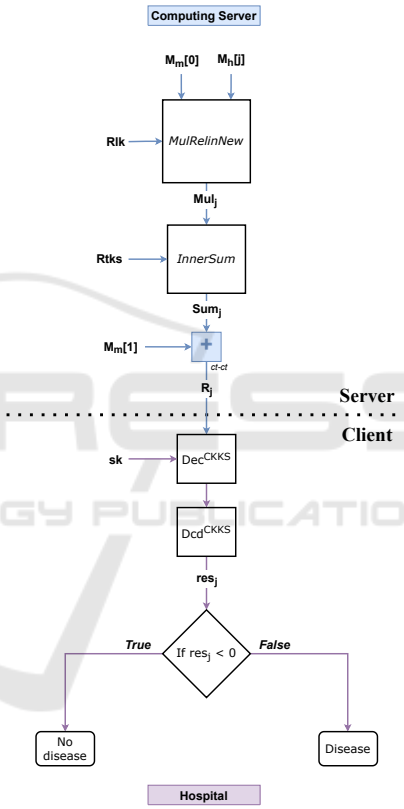


Figure 5: The HEALED encrypted inference and diagnosis result decryption after transciphering of patient data and model parameters.

5.2 Security Analysis

The security of HEALED follows directly from the guarantees of its cryptographic layer and the honest-but-curious threat model described in Section 4. The system relies on two protection layers: (i) symmetric encryption using the HE-friendly cipher Rubato, and (ii) homomorphic encryption using BFV and CKKS, which are semantically secure under the RLWE assumption.

Under this model, all entities follow the protocol correctly but may attempt to infer additional information from the data they legitimately access. The *Computing Server* processes only ciphertexts and at no point possesses decryption keys, preventing access to plaintext data, model parameters, or results. The *Hospital* can only decrypt the final diagnosis results and never accesses encrypted model parameters, while the *Model Owner* does not access patient data or results. This separation of roles and keys ensures confidentiality of both data and model. A critical component of HEALED is the TTP, assumed to be fully trusted and limited to cryptographic setup and key distribution. This introduces a strong trust assumption, but is a design choice aimed at significantly reducing client-side computation, communication, and key management overhead. This assumption reflects realistic deployment scenarios in healthcare, where trusted authorities (e.g., national health services) manage cryptographic material. With this knowledge, the TTP does not access plaintexts or ciphertexts after key distribution.

External adversaries are mitigated through authenticated and encrypted communication channels, preventing eavesdropping and message tampering. Given the semantic security of the encryption schemes, intercepted ciphertexts do not leak information about the underlying plaintexts.

6 EXPERIMENTAL EVALUATION AND ANALYSIS

In this section, we evaluate the HEALED prototype and compare it against a baseline implementation using only HE for linear SVM inference.

6.1 Experimental Environment

All experiments were conducted on a MacBook Air with an Apple M3 chip and 16 GB RAM. The HE-only baseline was adapted from Al Badawi et al.(Al Badawi and Yusof, 2024), implemented using the CKKS scheme with the OpenFHE library(Al Badawi et al., 2022). The SVM model was trained in Python using scikit-learn (Pedregosa et al., 2011). For the hybrid solution, the implementation was based on the RtF-Transciphering framework with Rubato (Ha et al., 2022).

The Breast Cancer Wisconsin (Diagnostic) dataset was used, containing 569 samples with 30 real-valued features (Street et al., 1993). The dataset was split into 80% training and 20% testing and rescaled to the interval $[-1,1]$. To ensure compatibility with RtF-Transciphering framework requisites, data and model parameters were padded with zeros to match the ring dimension, thus creating ciphertexts with 2^{14} to 2^{16} features.

6.2 Baseline

To establish a reference point, we reproduced and extended the HE-only SVM classification system of Al Badawi et al., additionally encrypting the model parameters to ensure full model confidentiality. A plaintext 10-fold cross-validation was first performed to validate correctness, yielding performance comparable to the original work. The classifier was then evaluated under three settings: plaintext inference, encrypted inference with plaintext parameters, and fully encrypted inference. Average performance metrics across multiple runs are reported in Table 1. Encrypting model parameters does not affect classification performance, since inference consists solely of linear operations that CKKS supports without precision loss.

Table 1: Model performance metrics under different encryption setups. Values are proportions in the range $[0,1]$.

Metric	Plaintext	Plaintext Param	Ciphertext Param	Overhead (%)
Accuracy	0.9543 ± 0.02	0.9421 ± 0.02	0.9421 ± 0.02	-1.28
Precision	0.9548 ± 0.03	0.9452 ± 0.03	0.9452 ± 0.03	-1.01
Recall	0.9749 ± 0.02	0.9675 ± 0.03	0.9675 ± 0.03	-0.76
F1-score	0.9642 ± 0.01	0.9557 ± 0.01	0.9557 ± 0.01	-0.88

Beyond model performance, the baseline was also evaluated in terms of computation and communication costs, which are compared with HEALED in the next subsection.

6.3 Parameter Study and Results

We evaluated HEALED under multiple configurations by varying the ring dimension N and the maximum ciphertext modulus QP, which jointly determine security, bootstrapping support, and computational cost. Security levels were estimated using the Lattice Estimator [2] assuming a Gaussian error distribution with $\sigma = 3.2$. Configurations are denoted as *logN-QP-Param*, comparing setups with plaintext and encrypted model parameters. Low-QP configurations do not support bootstrapping, which impacts correct decryption and inference accuracy despite achieving 128-bit security. Increasing QP enables bootstrapping and restores accuracy comparable to plaintext evaluation, at the cost of reduced security. Increasing the ring dimension N improves security guaran-

[2]https://github.com/malb/lattice-estimator

Table 2: Model performance metrics across the five setups.

Metric	14-Low-Plain	14-Low-Cipher	14-High-Plain	14-High-Cipher	16-High-Plain
Accuracy	0.3684 ± 0.03	0.6316 ± 0.03	0.9748 ± 0.01	0.9748 ± 0.01	0.9790 ± 0.01
Precision	0.0000	0.6316 ± 0.03	0.9691 ± 0.01	0.9691 ± 0.01	0.9729 ± 0.01
Recall	0.0000	1.0000	0.9913 ± 0.01	0.9913 ± 0.01	0.9945 ± 0.01
F1-Score	0.0000	0.7738 ± 0.02	0.98 ± 0.01	0.98 ± 0.01	0.9835 ± 0.01

tees but substantially increases server-side computational cost.

Table 2 reports model performance across the evaluated configurations. The 14-High and 16-High setups achieve the best accuracy, with negligible differences between plaintext and encrypted parameters.

Communication and computation costs are compared per feature, since the baseline processes only 30 features per sample, while HEALED operates over ciphertexts with capacity N. Table 3 shows encrypted data sizes for both approaches. HEALED reduces encrypted patient data by up to 4370× and encrypted model parameters by approximately 2100× per value. Even considering encrypted results, HEALED achieves a reduction of 53× in the worst case.

Table 3: Comparison of encrypted data size per-feature by each entity.

Setup	Hospital Encrypted data (bytes)	Model Owner Encrypted parameters (bytes)	Computing Server Encrypted results (bytes)
Baseline	33380.12	587.72	16707.6
HEALED	7.64	0.28	313.21*

* Highest value (from 16-High-Plain).

Latency results are reported in Table 4. Client-side encryption time is reduced by factors ranging from 62× to over 5700×, while decryption time is reduced by up to 7000× per value.

Table 4: Comparison of average execution time per-feature for encryption and decryption.

Setup	Encryption (μs)		Decryption (μs)
	Hospital	Model Owner	Hospital
Baseline	102.67		292.33
14-Low-Plain	0.018	-	0.041
14-Low-Cipher	0.021	1.146	0.042
14-High-Plain	0.149	-	1.325
14-High-Cipher	0.079	4.29	1.229
16-High-Plain	1.668	-	3.808

The 16-High-Plain configuration represents the most balanced scenario, achieving a 3.9% accuracy improvement over the baseline, 4370× smaller encrypted patient data, 62× faster encryption, and 76× faster decryption per value. This setup increases key storage on the hospital side by 2.31×. Server-side costs are substantially higher than in the baseline, reaching approximately 120GB of memory usage and 22 minutes of runtime per batch. These costs are expected as transciphering operates over large ciphertext spaces determined by the ring dimension.

6.4 Limitations

Runtime constraints prevented evaluation of the 16-High configuration with encrypted model parameters. A simple linear SVM limits conclusions regarding more complex ML models. The encrypted workload processed during transciphering is large due to padding and ciphertext packing, but experiments with larger sample counts and batch processing were not conducted. Finally, the system assumes an honest-but-curious threat model and relies on a fully trusted TTP, which represents a potential security limitation.

7 CONCLUSION AND FUTURE WORK

In this paper, we presented an HHE-based medical diagnosis PPML framework that addresses the efficiency constraints of HE, the security necessities for sensitive patient data, and protects the intellectual property of ML models. Combining the symmetric cipher Rubato with the BFV and CKKS HE schemes, the proposed framework minimizes client-side computation costs. Experimental results confirmed that the HEALED prototype achieved higher accuracy compared to the HE-only baseline, reduced overall encrypted data size, and significantly accelerated client-side operations.

Future work may address limitations, such as running experiments on higher security configurations, extending the implementation to larger datasets and more complex models, and implementing authenticated and encrypted channels. Encrypted training would also enable end-to-end secure ML pipelines, pushing HHE from inference-focused applications. Additionally, reducing the trust placed on the TTP is an important direction for future work. Possible approaches include distributing key generation across multiple entities or multi-party key generation protocols.

ACKNOWLEDGEMENTS

This research was funded by the Itea Project PHRESH (#23010) and Project PHRESH (COMPETE2030-FEDER-01232900 – 17556) co-funded by Portugal 2030. Furthermore, this work also received funding from the project UID/00760/2025.

REFERENCES

Abdinasibfar, H., Nuoskala, C., and Michalas, A. (2025). The HHE land: Exploring the landscape of hybrid homomorphic encryption. Cryptology ePrint Archive, Paper 2025/071.

Acar, A., Aksu, H., Uluagac, A. S., and Conti, M. (2018). A survey on homomorphic encryption schemes: Theory and implementation. *ACM Comput. Surv.*, 51(4).

Al Badawi, A. et al. (2022). OpenFHE: Open-source fully homomorphic encryption library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC'22, New York, NY, USA. Association for Computing Machinery.

Al Badawi, A. and Yusof, M. F. (2024). Private pathological assessment via machine learning and homomorphic encryption. *BioData Mining*, 17.

Bajwa, J., Munir, U., Nori, A., and Williams, B. (2021). Artificial intelligence in healthcare: transforming the practice of medicine. *Future Healthc. J.*, 8(2).

Brakerski, Z., Gentry, C., and Vaikuntanathan, V. (2012). (leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS '12, New York, NY, USA. Association for Computing Machinery.

Cheon, J. H., Kim, A., Kim, M., and Song, Y. (2017). Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, Lecture notes in computer science. Springer International Publishing, Cham.

Chillotti, I., Gama, N., Georgieva, M., and Izabachène, M. (2020). TFHE: Fast fully homomorphic encryption over the torus. *J. Cryptology*, 33(1).

Cho, J., Ha, J., Kim, S., Lee, B., Lee, J., Lee, J., Moon, D., and Yoon, H. (2021). Transciphering framework for approximate homomorphic encryption. In *Lecture Notes in Computer Science*, Lecture notes in computer science. Springer International Publishing, Cham.

Cong, K., Das, D., Park, J., and Pereira, H. V. (2022). Sortinghat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, New York, NY, USA. Association for Computing Machinery.

Dobraunig, C., Grassi, L., Helminger, L., Rechberger, C., Schofnegger, M., and Walch, R. (2023). Pasta: A case for hybrid homomorphic encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3).

Fan, J. and Vercauteren, F. (2012). Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144.

Frimpong, E., Nguyen, K., Budzys, M., Khan, T., and Michalas, A. (2024). GuardML: Efficient privacy-preserving machine learning services through hybrid homomorphic encryption.

Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, STOC '09, New York, NY, USA. Association for Computing Machinery.

Ha, J., Kim, S., Lee, B., Lee, J., and Son, M. (2022). Rubato: Noisy ciphers for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2022: 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 – June 3, 2022, Proceedings, Part I*, Berlin, Heidelberg. Springer-Verlag.

Huang, R. and Fang, S. (2024). Machine learning and big data analytics in the cloud environment. *International Journal of Cloud Computing and Database Management*, 5.

Huang, Y. and Duan, G. (2024). A privacy-preserving decision tree evaluation scheme for multiple wearable devices. In *2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*.

Méaux, P., Carlet, C., Journault, A., and Standaert, F.-X. (2019). Improved filter permutators for efficient FHE: Better instances and implementations. In *Lecture Notes in Computer Science*, Lecture notes in computer science. Springer International Publishing, Cham.

Moody, G. and Mark, R. (2001). The impact of the MIT-BIH arrhythmia database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3).

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12.

Podschwadt, R., Takabi, D., Hu, P., Rafiei, M. H., and Cai, Z. (2022). A survey of deep learning architectures for privacy-preserving machine learning with fully homomorphic encryption. *IEEE Access*, 10.

Saw, S. N. and Ng, K. H. (2022). Current challenges of implementing artificial intelligence in medical imaging. *Phys. Med.*, 100.

Street, W. N., Wolberg, W. H., and Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. In Acharya, R. S. and Goldgof, D. B., editors, *Biomedical Image Processing and Biomedical Visualization*. SPIE.

Tanuwidjaja, H. C., Choi, R., Baek, S., and Kim, K. (2020). Privacy-preserving deep learning on machine learning as a service—a comprehensive survey. *IEEE Access*, 8.